

Chip in, big win

UVM Sequence 常考题 / 真题汇总

Part 3 / 3 · 题目 17 ~ 24

SystemVerilog · UVM 1.2 / IEEE 1800.2

本册涵盖题目 17-24：ALU 溢出+饱和运算（题17）、FIFO 并发读写竞争（题18）、寄存器 SET/CLEAR/TOGGLE（题19）、网络协议 FSM 命令顺序（题20）、流水线气泡+吞吐率断言（题21）、PCIe Tag 回收池管理（题22）、AHB Burst 全模式+WRAP 算法（题23）、APB4 全特性（题24）。

题目 17　ALU 溢出标志 + 饱和/回绕运算验证

难度：中级
标签：ALU / overflow / saturate / 参考模型 / 定向边界值 / 四则运算

【题目背景】

16-bit 无符号 ALU 支持 ADD/SUB/MUL/DIV 四种运算。 当运算结果超出 16-bit 无符号范围（> 0xFFFF）时，产生溢出。 ALU 支持两种溢出处理模式（通过 saturate_mode 选择）： - 饱和模式（saturate=1）：结果钳位到最大/最小值（0xFFFF / 0x0000）； - 回绕模式（saturate=0）：结果保留低 16 位（即取模）。

【完整要求】

Phase 1（随机测试，num_random=20 次）： Phase 2（定向溢出用例，num_directed=8 次）：

- 随机化 opcode / operand_a / operand_b / saturate_mode；
- 约束 DIV 时 operand_b != 0（避免除以零）；
- 调用 req.expected_result(exp_ovf) 计算期望值；
- 比对 rsp.result 与 exp_res，不一致则 `uvm_error；
- 约束 opcode==ALU_ADD, operand_a==16'hFFFF, operand_b>0；
- saturate_mode 在 0 和 1 之间各 50% 分布；
- 断言 rsp.overflow==1（必须检测到溢出）；
- 断言 rsp.result 符合参考模型（饱和/回绕各自正确）。

【实现要点】

> - expected_result() 函数是本题核心，实现 4 种运算 × 2 种模式 = 8 种组合；
> - 饱和 ADD：16'hFFFF + 1 → 饱和模式=0xFFFF，回绕模式=0x0000；
> - 定向测试中 operand_a==16'hFFFF 是边界值，确保任意正 operand_b 都溢出。

【完整答案代码（含全部类定义）】

```
class alu_transaction extends uvm_sequence_item;
    typedef enum bit [1:0] { ALU_ADD, ALU_SUB, ALU_MUL, ALU_DIV } alu_op_e;

    rand alu_op_e   opcode;
    rand bit [15:0] operand_a, operand_b;
    rand bit        saturate_mode;   // 考点: 饱和/回绕选择
    bit      [31:0] result;
    bit             overflow;

    `uvm_object_utils_begin(alu_transaction)
        `uvm_field_enum(alu_op_e, opcode, UVM_ALL_ON)
        `uvm_field_int(operand_a,         UVM_ALL_ON)
        `uvm_field_int(operand_b,         UVM_ALL_ON)
        `uvm_field_int(saturate_mode,     UVM_ALL_ON)
        `uvm_field_int(result,            UVM_ALL_ON)
        `uvm_field_int(overflow,          UVM_ALL_ON)
    `uvm_object_utils_end

    constraint div_safe_c { opcode == ALU_DIV -> operand_b != 0; }

    function new(string name = "alu_transaction");
        super.new(name);
    endfunction

    function bit [31:0] expected_result(output bit exp_ovf);
        bit [32:0] ext;
        exp_ovf = 0;
        case (opcode)
            ALU_ADD: begin
                ext = {1'b0,operand_a}+{1'b0,operand_b};
                exp_ovf = ext[16];
                if (saturate_mode && exp_ovf) return 32'h0000_FFFF;
                return {16'h0,ext[15:0]};
            end
            ALU_SUB: begin
                ext = {1'b0,operand_a}-{1'b0,operand_b};
                exp_ovf = ext[16];
                if (saturate_mode && exp_ovf) return 32'h0;
                return {16'h0,ext[15:0]};
            end
            ALU_MUL: begin
                bit [31:0] prod = operand_a * operand_b;
                exp_ovf = (prod > 32'hFFFF);
                if (saturate_mode && exp_ovf) return 32'h0000_FFFF;
                return prod;
            end
            ALU_DIV: return operand_a / operand_b;
        endcase
    endfunction
endclass

class alu_overflow_sequence extends uvm_sequence#(alu_transaction);
    `uvm_object_utils(alu_overflow_sequence)

    int unsigned num_random, num_directed;

    function new(string name = "alu_overflow_sequence");
        super.new(name);
        num_random=20; num_directed=8;
    endfunction

    task body();
        alu_transaction req, rsp;
        bit exp_ovf; bit [31:0] exp_res;

        repeat (num_random) begin
            req = alu_transaction::type_id::create("rnd");
            start_item(req);
            if (!req.randomize()) `uvm_fatal("RAND_FAIL","ALU rand failed")
            finish_item(req); get_response(rsp);
            exp_res = req.expected_result(exp_ovf);
            if (rsp.result !== exp_res)
```

```
                `uvm_error("ALU_OVF",
                    $sformatf("op=%s sat=%0b exp=0x%08h got=0x%08h",
                        req.opcode.name(),req.saturate_mode,exp_res,rsp.result))
        end

        // 考点：强制溢出边界用例
        repeat (num_directed) begin
            req = alu_transaction::type_id::create("ovf");
            start_item(req);
            if (!req.randomize() with {
                opcode      == ALU_ADD;
                operand_a   == 16'hFFFF;
                operand_b   inside {[16'h1:16'hFFFF]};
                saturate_mode dist {0:=50, 1:=50};
            }) `uvm_fatal("RAND_FAIL","ALU directed rand failed")
            finish_item(req); get_response(rsp);
            exp_res = req.expected_result(exp_ovf);
            if (!rsp.overflow)
                `uvm_error("ALU_OVF","Overflow flag not set on boundary ADD")
            if (rsp.result !== exp_res)
                `uvm_error("ALU_OVF",
                    $sformatf("Saturate=%0b exp=0x%08h got=0x%08h",
                        req.saturate_mode,exp_res,rsp.result))
            else
                `uvm_info("ALU_OVF",
                    $sformatf("Overflow OK sat=%0b result=0x%08h",
                        req.saturate_mode,rsp.result), UVM_MEDIUM)
        end
    endtask
endclass
```

题目 18　FIFO 同时读写（并发）竞争条件验证

难度：高级
标签：FIFO / concurrent / wr_en / rd_en / reference model / count / 竞争

【题目背景】

同步 FIFO 通常支持"同周期同时读写"（simultaneous read-write）。 当 wr_en 和 rd_en 同时为 1 时，FIFO 的行为由实现决定： 通常是先写后读（write-through），即写入的数据可以在同周期被读出； 或先读后写（read-first），即读出旧数据，同时写入新数据。 DUT 的 count 计数器也需要在并发情况下保持正确（先写后读：count 不变）。

【完整要求】

- 1. 约束 at_least_one_c：wr_en || rd_en（至少有一个操作有效）；
- 2. 约束 concurrent_c：(wr_en && rd_en) dist {0:=80, 1:=20}（20% 并发）；
- 3. 动态约束：满时禁止 wr_en，空时禁止 rd_en；
- 4. 参考模型队列 ref_q[$] 按"先写后读"语义更新：
- wr_en==1：push_back(wdata)；
- rd_en==1 && ref_q.size()>0：pop_front()，比对 rsp.rdata；
- 5. 每周期断言 rsp.count == ref_q.size()；
- 6. 运行 num_ops=64 个操作周期。

【实现要点】

> - 动态约束（满/空时禁止对应操作）在 randomize() with {} 块中实现；
> - 参考模型队列是本题正确性的核心，必须与 DUT 的读写语义（先写后读）保持一致；
> - count 校验是比数据校验更细粒度的结构验证，能发现 count 逻辑的 bug。

【完整答案代码（含全部类定义）】

```
class fifo_concurrent_item extends uvm_sequence_item;
    rand bit        wr_en, rd_en;
    rand bit [31:0] wdata;
         bit [31:0] rdata;
         bit        full, empty;
         bit [5:0]  count;   // driver 回填

    `uvm_object_utils_begin(fifo_concurrent_item)
        `uvm_field_int(wr_en, UVM_ALL_ON)
        `uvm_field_int(rd_en, UVM_ALL_ON)
        `uvm_field_int(wdata, UVM_ALL_ON)
        `uvm_field_int(rdata, UVM_ALL_ON)
        `uvm_field_int(full,  UVM_ALL_ON)
        `uvm_field_int(empty, UVM_ALL_ON)
        `uvm_field_int(count, UVM_ALL_ON)
    `uvm_object_utils_end

    constraint at_least_one_c { wr_en || rd_en; }
    // 考点: 20% 并发概率
    constraint concurrent_c   { (wr_en && rd_en) dist {0:=80, 1:=20}; }

    function new(string name = "fifo_concurrent_item");
        super.new(name);
    endfunction
endclass

class fifo_concurrent_sequence extends uvm_sequence#(fifo_concurrent_item);
    `uvm_object_utils(fifo_concurrent_sequence)

    localparam int FIFO_DEPTH = 32;
    int unsigned   num_ops;

    function new(string name = "fifo_concurrent_sequence");
        super.new(name);
        num_ops = 64;
    endfunction

    task body();
        fifo_concurrent_item req, rsp;
        bit [31:0] ref_queue[$];

        repeat (num_ops) begin
            req = fifo_concurrent_item::type_id::create("conc");
            start_item(req);
            if (!req.randomize() with {
                wr_en -> (ref_queue.size() < FIFO_DEPTH);
                rd_en -> (ref_queue.size() > 0);
            }) `uvm_fatal("RAND_FAIL","concurrent rand failed")
            finish_item(req); get_response(rsp);

            // 参考模型: 先写后读
            if (req.wr_en) ref_queue.push_back(req.wdata);
            if (req.rd_en && ref_queue.size() > 0) begin
                bit [31:0] exp = ref_queue.pop_front();
                if (rsp.rdata !== exp)
                    `uvm_error("CONC_SEQ",
                        $sformatf("Data mismatch: exp=0x%08h got=0x%08h",exp,rsp.rdata))
            end
            // 考点: count 校验
            if (rsp.count !== ref_queue.size())
                `uvm_error("CONC_SEQ",
                    $sformatf("Count mismatch: exp=%0d got=%0d",ref_queue.size(),rsp.count))
        end
        `uvm_info("CONC_SEQ","Concurrent R/W test complete",UVM_LOW)
    endtask
endclass
```

题目 19　寄存器 SET / CLEAR / TOGGLE 三种原子操作验证

难度：中级
标签：register / SET / CLEAR / TOGGLE / atomic / shadow model

【题目背景】

某些高性能 IP 的寄存器接口支持三种原子写操作模式， 通过访问地址的高位区分（例如 addr[13:12] 决定操作类型）： - SET 模式（OP_SET） ：写 1 置位，写 0 无效； - CLEAR 模式（OP_CLEAR）：写 1 清除，写 0 无效； - TOGGLE 模式（OP_TOGGLE）：写 1 翻转，写 0 无效； - NORMAL 模式（OP_NORMAL）：普通读写。 这些原子操作可以避免"读-改-写"的竞争条件。

【完整要求】

使用 shadow 寄存器（bit[31:0] shadow）跟踪期望值， 每步操作后读回 DUT 并与 shadow 比对： Step 0（初始化）：NORMAL 写入 32'hA5A5_A5A5，shadow = 0xA5A5_A5A5； Step 1（SET）：mask=32'h0000_00FF，OP_SET 写入，shadow |= mask； NORMAL 读回，断言与 shadow 一致； Step 2（CLEAR）：mask=32'h0000_FF00，OP_CLEAR 写入，shadow &= ~mask； NORMAL 读回，断言与 shadow 一致； Step 3（TOGGLE）：mask=32'hFFFF_0000，OP_TOGGLE 写入，shadow ^= mask； NORMAL 读回，断言与 shadow 一致。

【实现要点】

- do_op() 辅助 task 封装 start_item/finish_item，使 body() 逻辑清晰；
- shadow 寄存器与每步操作同步更新，是验证原子操作正确性的关键；
- 三种操作覆盖了位操作的完整语义，缺一不可。

【完整答案代码（含全部类定义）】

```
class reg_atomic_item extends uvm_sequence_item;
    typedef enum bit [1:0] {
        OP_NORMAL=2'b00, OP_SET=2'b01, OP_CLEAR=2'b10, OP_TOGGLE=2'b11
    } reg_op_e;

    rand reg_op_e   op_mode;
    rand bit [11:0] addr;
    rand bit [31:0] wdata;
    rand bit        is_write;
         bit [31:0] rdata;

    `uvm_object_utils_begin(reg_atomic_item)
        `uvm_field_enum(reg_op_e, op_mode, UVM_ALL_ON)
        `uvm_field_int(addr,               UVM_ALL_ON)
        `uvm_field_int(wdata,              UVM_ALL_ON)
        `uvm_field_int(is_write,           UVM_ALL_ON)
        `uvm_field_int(rdata,              UVM_ALL_ON)
    `uvm_object_utils_end

    constraint addr_align_c { addr[1:0] == 2'b00; }

    function new(string name = "reg_atomic_item");
        super.new(name);
    endfunction
endclass

class reg_atomic_ops_sequence extends uvm_sequence#(reg_atomic_item);
    `uvm_object_utils(reg_atomic_ops_sequence)

    localparam bit [11:0] TEST_ADDR = 12'h020;

    function new(string name = "reg_atomic_ops_sequence");
        super.new(name);
    endfunction

    task do_op(reg_atomic_item::reg_op_e mode, bit is_wr,
               bit [31:0] wd, output bit [31:0] rd);
        reg_atomic_item req, rsp;
        req = reg_atomic_item::type_id::create("atomic");
        start_item(req);
        req.op_mode=mode; req.addr=TEST_ADDR; req.is_write=is_wr; req.wdata=wd;
        finish_item(req); get_response(rsp);
        rd = rsp.rdata;
    endtask

    task body();
        bit [31:0] shadow = 32'h0, mask, rdata;

        // 初始化
        do_op(reg_atomic_item::OP_NORMAL, 1, 32'hA5A5_A5A5, rdata);
        shadow = 32'hA5A5_A5A5;

        // 考点: SET
        mask = 32'h0000_00FF;
        do_op(reg_atomic_item::OP_SET, 1, mask, rdata);
        shadow = shadow | mask;
        do_op(reg_atomic_item::OP_NORMAL, 0, 0, rdata);
        if (rdata !== shadow) `uvm_error("ATOMIC",$sformatf("SET mismatch exp=0x%08h got=0x%08h",shadow,rdata))
        else `uvm_info("ATOMIC","SET OK",UVM_MEDIUM)

        // 考点: CLEAR
        mask = 32'h0000_FF00;
        do_op(reg_atomic_item::OP_CLEAR, 1, mask, rdata);
        shadow = shadow & ~mask;
        do_op(reg_atomic_item::OP_NORMAL, 0, 0, rdata);
        if (rdata !== shadow) `uvm_error("ATOMIC",$sformatf("CLEAR mismatch exp=0x%08h got=0x%08h",shadow,rdata))
        else `uvm_info("ATOMIC","CLEAR OK",UVM_MEDIUM)

        // 考点: TOGGLE
        mask = 32'hFFFF_0000;
        do_op(reg_atomic_item::OP_TOGGLE, 1, mask, rdata);
        shadow = shadow ^ mask;
```

```
        do_op(reg_atomic_item::OP_NORMAL, 0, 0, rdata);
        if (rdata !== shadow) `uvm_error("ATOMIC",$sformatf("TOGGLE mismatch exp=0x%08h got=0x%08h",shadow,rdata))
        else `uvm_info("ATOMIC","TOGGLE OK",UVM_LOW)
    endtask
endclass
```

题目 20　网络协议 FSM 合法命令顺序约束

难度：中级
标签：network protocol / FSM / command order / CONNECT / DISCONNECT / 合法序列

【题目背景】

DUT 实现了一个简单的网络连接协议状态机，强制命令执行顺序： IDLE → CONNECT → DATA_TRANSFER → DISCONNECT → IDLE 其中 DATA_TRANSFER 阶段可以执行任意次数的 SEND 或 RECEIVE 命令。 非法顺序（如未 CONNECT 就 SEND，或重复 CONNECT）应被 DUT 拒绝 并返回非零 status。 验证目标：覆盖"直接确认/无数据/有数据"等多种合法路径， 并验证 DUT 对合法命令序列的正确响应。

【完整要求】

固定 cmd_type，其余字段随机化，发送后 get_response； a. CONNECT：若 status!=0 则 `uvm_error 并 continue（跳过本轮）； b. 随机 data_cmds 次（1~max_data_cmds），每次随机选 SEND 或 RECEIVE； 若 status!=0 则 `uvm_error； c. DISCONNECT：若 status!=0 则 `uvm_error；

- 1. 实现 send_cmd(cmd_type, rsp_out) 辅助 task：
- 2. 外层 repeat(num_sessions) 驱动多轮完整会话；
- 3. 每轮严格按顺序执行：
- 4. 成功完成一轮则 `uvm_info 打印本轮数据命令数。

【实现要点】

> - send_cmd() 中固定 cmd_type、随机其余字段：体现"定向+随机"混合激励；
> - void'(std::randomize(data_cmd) with {data_cmd inside {SEND, RECEIVE}}) 随机选操作；
> - continue 语句在 CONNECT 失败时跳过后续步骤，避免无效的 SEND/DISCONNECT。

【完整答案代码（含全部类定义）】

```
class net_transaction extends uvm_sequence_item;
    typedef enum bit [2:0] {
        NET_CONNECT=3'b000, NET_SEND=3'b001,
        NET_RECEIVE=3'b010, NET_DISCONNECT=3'b011, NET_PING=3'b100
    } net_cmd_e;

    rand net_cmd_e  cmd_type;
    rand bit [31:0] dst_ip;
    rand bit [15:0] dst_port;
    rand bit [63:0] payload;
    rand bit [7:0]  payload_len;
    bit      [7:0]  status;

    `uvm_object_utils_begin(net_transaction)
        `uvm_field_enum(net_cmd_e, cmd_type, UVM_ALL_ON)
        `uvm_field_int(dst_ip,              UVM_ALL_ON)
        `uvm_field_int(dst_port,            UVM_ALL_ON)
        `uvm_field_int(payload,             UVM_ALL_ON)
        `uvm_field_int(payload_len,         UVM_ALL_ON)
        `uvm_field_int(status,              UVM_ALL_ON)
    `uvm_object_utils_end

    constraint len_c { payload_len inside {[1:64]}; }

    function new(string name = "net_transaction");
        super.new(name);
    endfunction
endclass

class net_protocol_fsm_sequence extends uvm_sequence#(net_transaction);
    `uvm_object_utils(net_protocol_fsm_sequence)

    int unsigned num_sessions, max_data_cmds;

    function new(string name = "net_protocol_fsm_sequence");
        super.new(name);
        num_sessions=4; max_data_cmds=6;
    endfunction

    task send_cmd(net_transaction::net_cmd_e cmd, output net_transaction rsp_out);
        net_transaction req;
        req = net_transaction::type_id::create("net_txn");
        start_item(req);
        // 考点: 固定 cmd_type，其余字段随机化
        if (!req.randomize() with { cmd_type == cmd; })
            `uvm_fatal("RAND_FAIL","Net cmd rand failed")
        finish_item(req); get_response(rsp_out);
    endtask

    task body();
        net_transaction rsp;
        int data_cmds;
        net_transaction::net_cmd_e data_cmd;

        repeat (num_sessions) begin
            // 考点: 严格 CONNECT 先行
            send_cmd(net_transaction::NET_CONNECT, rsp);
            if (rsp.status != 8'h00) begin
                `uvm_error("NET_FSM",$sformatf("CONNECT failed: 0x%02h",rsp.status))
                continue;
            end

            void'(std::randomize(data_cmds) with {data_cmds inside {[1:max_data_cmds]};});
            repeat (data_cmds) begin
                void'(std::randomize(data_cmd) with {
                    data_cmd inside {net_transaction::NET_SEND, net_transaction::NET_RECEIVE};
                });
                send_cmd(data_cmd, rsp);
                if (rsp.status != 8'h00)
                    `uvm_error("NET_FSM",
                        $sformatf("%s failed: 0x%02h",data_cmd.name(),rsp.status))
            end
```

```
            // 考点: DISCONNECT 结束会话
            send_cmd(net_transaction::NET_DISCONNECT, rsp);
            if (rsp.status != 8'h00)
                `uvm_error("NET_FSM","DISCONNECT failed")
            else
                `uvm_info("NET_FSM",
                    $sformatf("Session done: %0d data cmds",data_cmds), UVM_MEDIUM)
        end
    endtask
endclass
```

题目 21　流水线气泡（Bubble）注入 + 吞吐率阈值断言

难度：高级
标签：pipeline / bubble / NOP / throughput / valid_out / 吞吐率

【题目背景】

流水线处理器在指令间可以插入气泡（NOP/bubble）来处理数据冒险、　控制冒险等情况，代价是降低有效吞吐率。　验证目标：　① 在混有 NOP 气泡的激励下，DUT 不丢失有效数据（非 NOP 拍的输出必须有效）；　② 统计实际吞吐率，验证其不低于设计预期的最低阈值。

【完整要求】

- 1. is_nop 的 constraint：dist { 0:=70, 1:=30 }（30% 气泡率）；
- 2. DATA 拍（is_nop==0）由序列层赋予递增 txn_id；
- 3. NOP 拍不赋 txn_id（值无关）；
- 4. 循环 num_cycles=50 个周期，每周期计数 total，若 rsp.valid_out==1 则 valid_cnt++；
- 5. 全部完成后计算吞吐率：tp = real'(valid_cnt) / real'(total)；
- 6. 打印 `uvm_info 报告吞吐率和计数；
- 7. 若 tp < min_throughput（默认 0.5），打印 `uvm_error。

【实现要点】

> - 气泡率 30%，理论吞吐率约 0.7（DATA 占 70%），min_throughput=0.5 留有余量；
> - real 类型转换：real'(int_val) 是 SystemVerilog 的类型强制转换；
> - 本题是性能指标验证的典型模型，可扩展为功能覆盖率与性能的联合分析。

【完整答案代码（含全部类定义）】

```
class pipeline_transaction extends uvm_sequence_item;
    rand bit        is_nop;      // 考点: 气泡控制
    rand bit [31:0] data_in;
    rand bit [7:0]  txn_id;
    bit      [31:0] data_out;
    bit      [7:0]  out_id;
    bit             valid_out;

    `uvm_object_utils_begin(pipeline_transaction)
        `uvm_field_int(is_nop,    UVM_ALL_ON)
        `uvm_field_int(data_in,   UVM_ALL_ON)
        `uvm_field_int(txn_id,    UVM_ALL_ON)
        `uvm_field_int(data_out,  UVM_ALL_ON)
        `uvm_field_int(out_id,    UVM_ALL_ON)
        `uvm_field_int(valid_out, UVM_ALL_ON)
    `uvm_object_utils_end

    constraint bubble_rate_c { is_nop dist {0:=70, 1:=30}; }

    function new(string name = "pipeline_transaction");
        super.new(name);
    endfunction
endclass

class pipeline_bubble_sequence extends uvm_sequence#(pipeline_transaction);
    `uvm_object_utils(pipeline_bubble_sequence)

    int unsigned num_cycles;
    real         min_throughput;

    function new(string name = "pipeline_bubble_sequence");
        super.new(name);
        num_cycles=50; min_throughput=0.5;
    endfunction

    task body();
        pipeline_transaction req, rsp;
        int valid_cnt=0, total=0, id_cnt=0;

        repeat (num_cycles) begin
            req = pipeline_transaction::type_id::create("pipe");
            start_item(req);
            if (!req.randomize()) `uvm_fatal("RAND_FAIL","pipe rand failed")
            if (!req.is_nop) req.txn_id = id_cnt++;
            finish_item(req); get_response(rsp);
            total++;
            if (rsp.valid_out) valid_cnt++;
        end

        // 考点: 吞吐率断言
        begin
            real tp = real'(valid_cnt)/real'(total);
            `uvm_info("PIPE_SEQ",
                $sformatf("Throughput=%.2f (%0d/%0d)",tp,valid_cnt,total), UVM_LOW)
            if (tp < min_throughput)
                `uvm_error("PIPE_SEQ",
                    $sformatf("Throughput %.2f below %.2f",tp,min_throughput))
        end
    endtask
endclass
```

题目 22　PCIe TLP Tag 回收池管理（Tag Recycling）

难度：高级
标签：PCIe / TLP / Tag recycling / tag pool / Non-Posted / bit vector

【题目背景】

PCIe 协议规定：Non-Posted 请求（如 Memory Read，TLP_MEM_RD）必须携带 唯一的 Tag 字段（5-bit，共 32 个可用值：0~31）。 - Tag 在请求发出时"占用"； - Requester 收到对应 Completion TLP 后，Tag 才可被"回收"再用； - 在 Completion 到达之前，相同 Tag 不得发出第二次请求。 Tag 管理错误是 PCIe IP 验证中的高频 bug，可能导致响应被错误路由。

【完整要求】

- 1. 定义 bit[31:0] tag_pool = 32'hFFFF_FFFF（全部空闲）；
- 2. 发出请求前，for 循环在 tag_pool 中找第一个空闲位（tag_pool[t]==1）：
- 记录 allocated_tag = t；
- 置 tag_pool[t] = 0（标记占用）；
- 若无空闲位则 `uvm_fatal "Tag pool exhausted"；
- 3. 仅约束 tlp_type==TLP_MEM_RD，其余字段随机化；
- 4. 手动设置 req.tag = allocated_tag；
- 5. 收到 get_response 后：
- 断言 rsp.tag == allocated_tag（ID 匹配）；
- 断言 tag_pool[rsp.tag] == 0（确认 Tag 在归还前未被重分配）；
- 执行归还：tag_pool[rsp.tag] = 1；
- 若 cpl_status != 3'b000 则 `uvm_error（非 SC Completion）；
- 6. 全部完成后打印 final_pool 状态。

【实现要点】

> - tag 字段不使用 randomize，由序列层完全控制，是本题与普通随机测试的区别；
> - bit 向量位操作（pool[t]）是 SystemVerilog 的基本功；
> - "未归还前不得重分配"的断言是 Tag 管理正确性的核心保障。

【完整答案代码（含全部类定义）】

```
class pcie_axi_transaction extends uvm_sequence_item;
    typedef enum bit [1:0] {
        TLP_MEM_WR=2'b00, TLP_MEM_RD=2'b01,
        TLP_CFG_WR=2'b10, TLP_CFG_RD=2'b11
    } tlp_type_e;

    rand tlp_type_e  tlp_type;
    rand bit [63:0]  addr;
    rand bit [9:0]   length_dw;
    rand bit [4:0]   tag;        // 由序列层手动分配
    rand bit [2:0]   tc;
    bit      [31:0]  rdata[];
    bit      [2:0]   cpl_status;

    `uvm_object_utils_begin(pcie_axi_transaction)
        `uvm_field_enum(tlp_type_e, tlp_type, UVM_ALL_ON)
        `uvm_field_int(addr,                  UVM_ALL_ON)
        `uvm_field_int(length_dw,             UVM_ALL_ON)
        `uvm_field_int(tag,                   UVM_ALL_ON)
        `uvm_field_int(tc,                    UVM_ALL_ON)
        `uvm_field_int(cpl_status,            UVM_ALL_ON)
    `uvm_object_utils_end

    constraint burst_c { length_dw inside {10'd1,10'd4,10'd8}; }
    constraint align_c { addr[1:0] == 2'b00; }
    constraint tc_c    { tc inside {[0:7]}; }

    function new(string name = "pcie_axi_transaction");
        super.new(name);
    endfunction
endclass

class pcie_tag_recycling_sequence extends uvm_sequence#(pcie_axi_transaction);
    `uvm_object_utils(pcie_tag_recycling_sequence)

    localparam int TAG_POOL_SIZE = 32;
    int unsigned   num_reads;

    function new(string name = "pcie_tag_recycling_sequence");
        super.new(name);
        num_reads = 40;
    endfunction

    task body();
        pcie_axi_transaction req, rsp;
        // 考点: Tag 池 bit 向量
        bit [31:0] tag_pool = 32'hFFFF_FFFF;
        int allocated_tag;

        repeat (num_reads) begin
            // 从池中取一个空闲 Tag
            allocated_tag = -1;
            for (int t = 0; t < TAG_POOL_SIZE; t++) begin
                if (tag_pool[t]) begin
                    allocated_tag = t;
                    tag_pool[t]   = 0;   // 标记占用
                    break;
                end
            end
            if (allocated_tag == -1)
                `uvm_fatal("PCIE_TAG","Tag pool exhausted!")

            req = pcie_axi_transaction::type_id::create("pcie_rd");
            start_item(req);
            if (!req.randomize() with { tlp_type == pcie_axi_transaction::TLP_MEM_RD; })
                `uvm_fatal("RAND_FAIL","PCIe rand failed")
            req.tag = allocated_tag;   // 考点: 序列层手动指定 Tag
            finish_item(req); get_response(rsp);

            if (rsp.tag !== allocated_tag)
                `uvm_error("PCIE_TAG",
                    $sformatf("Tag mismatch: sent=%0d got=%0d",allocated_tag,rsp.tag))
```

```
            if (tag_pool[rsp.tag] != 0)
                `uvm_error("PCIE_TAG",
                    $sformatf("Tag %0d re-used before recycle!",rsp.tag))

            // 归还 Tag
            tag_pool[rsp.tag] = 1;
            `uvm_info("PCIE_TAG",
                $sformatf("Tag %0d recycled pool=0x%08h",rsp.tag,tag_pool), UVM_HIGH)
            if (rsp.cpl_status != 3'b000)
                `uvm_error("PCIE_TAG",
                    $sformatf("Non-SC completion: tag=%0d status=%03b",rsp.tag,rsp.cpl_status))
        end
        `uvm_info("PCIE_TAG",
            $sformatf("Tag recycling done, final_pool=0x%08h",tag_pool), UVM_LOW)
    endtask
endclass
```

题目 23　AHB-Lite Burst 全模式覆盖

难度：高级
标签：AHB-Lite / HBURST / HTRANS / BUSY / HREADY / wait state / WRAP / INCR

【题目背景】

AHB-Lite（AMBA 3 AHB-Lite Protocol Specification）定义了 8 种突发模式： SINGLE（单次传输）、INCR（不定长增量）、WRAP4/INCR4、WRAP8/INCR8、WRAP16/INCR16（定长回绕/增量） 以及 4 种传输类型：IDLE / BUSY / NONSEQ / SEQ。BUSY 周期允许 master 在突发中途暂停但保持总线仲裁权。 HREADY=0 时 slave 插入等待周期，总线保持 ACCESS 状态。

【完整要求】

- 1. 定义 allowed_bursts[$] 数组包含全部 8 种 hburst 类型；
- 2. 每轮突发：随机选择 chosen_burst，发送 NONSEQ 首拍（htrans=NONSEQ）；
- 3. 后续 beats-1 拍：
- 随机决定是否在当前位置插入 BUSY 周期（busy_remain 控制上限）；
- 发送 SEQ 拍，haddr 由 beat_addr() 参考模型计算；
- 4. 每拍收到响应后累计 wait_cycles 统计 HREADY 等待；
- 5. 若任一拍 rsp.hresp==1（ERROR），打印 `uvm_error 并 break；
- 6. 每次突发结束打印突发类型、beat 数、总等待周期；
- 7. WRAP 类型的对齐约束：WRAP4×WORD=16B，WRAP8×WORD=32B，WRAP16×WORD=64B。

【完整答案代码（含全部类定义）】

```
class ahb_burst_item extends uvm_sequence_item;
    typedef enum bit [1:0] {
        IDLE   = 2'b00,
        BUSY   = 2'b01,
        NONSEQ = 2'b10,
        SEQ    = 2'b11
    } htrans_e;

    typedef enum bit [2:0] {
        SINGLE = 3'b000,
        INCR   = 3'b001,
        WRAP4  = 3'b010,
        INCR4  = 3'b011,
        WRAP8  = 3'b100,
        INCR8  = 3'b101,
        WRAP16 = 3'b110,
        INCR16 = 3'b111
    } hburst_e;

    typedef enum bit [2:0] {
        BYTE     = 3'b000,
        HALFWORD = 3'b001,
        WORD     = 3'b010,
        DWORD    = 3'b011
    } hsize_e;

    rand htrans_e   htrans;
    rand hburst_e   hburst;
    rand hsize_e    hsize;
    rand bit        hwrite;
    rand bit [31:0] haddr;
    rand bit [31:0] hwdata;
    rand bit [2:0]  busy_slots;  // 突发中 BUSY 周期数 (0~4)
         bit [31:0] hrdata;
         bit        hresp;       // 0=OKAY, 1=ERROR (driver 回填)
         int        wait_cycles; // 实际等待周期数 (driver 回填)

    `uvm_object_utils_begin(ahb_burst_item)
        `uvm_field_enum(htrans_e, htrans,     UVM_ALL_ON)
        `uvm_field_enum(hburst_e, hburst,     UVM_ALL_ON)
        `uvm_field_enum(hsize_e,  hsize,      UVM_ALL_ON)
        `uvm_field_int(hwrite,                UVM_ALL_ON)
        `uvm_field_int(haddr,                 UVM_ALL_ON)
        `uvm_field_int(hwdata,                UVM_ALL_ON)
        `uvm_field_int(busy_slots,            UVM_ALL_ON)
        `uvm_field_int(hrdata,                UVM_ALL_ON)
        `uvm_field_int(hresp,                 UVM_ALL_ON)
        `uvm_field_int(wait_cycles,           UVM_ALL_ON)
    `uvm_object_utils_end

    // 序列首拍固定 NONSEQ; SINGLE 不允许 BUSY
    constraint default_nonseq_c { htrans == NONSEQ; }
    constraint single_no_busy_c { hburst == SINGLE -> busy_slots == 0; }
    constraint busy_range_c     { busy_slots inside {[0:4]}; }
    constraint addr_range_c     { haddr <= 32'hFFFF_FFC0; }

    // WRAP 对齐约束 (WORD 访问)
    constraint wrap4_align_c  { (hburst==WRAP4  && hsize==WORD) -> haddr[3:0]==4'h0; }
    constraint wrap8_align_c  { (hburst==WRAP8  && hsize==WORD) -> haddr[4:0]==5'h0; }
    constraint wrap16_align_c { (hburst==WRAP16 && hsize==WORD) -> haddr[5:0]==6'h0; }

    function new(string name = "ahb_burst_item");
        super.new(name);
    endfunction

    function int burst_beats();
        case (hburst)
            SINGLE:             return 1;
            INCR:               return 4;
            WRAP4,  INCR4:      return 4;
            WRAP8,  INCR8:      return 8;
            WRAP16, INCR16:     return 16;
```

```
            default:           return 1;
        endcase
    endfunction

    // WRAP/INCR 通用地址计算参考模型
    function bit [31:0] beat_addr(int n, bit [31:0] base);
        int bytes = 1 << int'(hsize);
        bit [31:0] wrap_mask;
        case (hburst)
            WRAP4:  wrap_mask = (4  * bytes) - 1;
            WRAP8:  wrap_mask = (8  * bytes) - 1;
            WRAP16: wrap_mask = (16 * bytes) - 1;
            default: return base + n * bytes;
        endcase
        return (base & ~wrap_mask) | ((base + n * bytes) & wrap_mask);
    endfunction
endclass

class ahb_burst_sequence extends uvm_sequence#(ahb_burst_item);
    `uvm_object_utils(ahb_burst_sequence)

    int unsigned num_bursts = 12;
    ahb_burst_item::hburst_e allowed_bursts[$];

    function new(string name = "ahb_burst_sequence");
        super.new(name);
        // 默认覆盖全部 8 种突发类型
        allowed_bursts = '{
            ahb_burst_item::SINGLE,
            ahb_burst_item::INCR,
            ahb_burst_item::WRAP4,  ahb_burst_item::INCR4,
            ahb_burst_item::WRAP8,  ahb_burst_item::INCR8,
            ahb_burst_item::WRAP16, ahb_burst_item::INCR16
        };
    endfunction

    task body();
        ahb_burst_item req, rsp;
        ahb_burst_item::hburst_e chosen_burst;
        int beats, busy_remain, total_wait;
        bit [31:0] exp_addr;

        repeat (num_bursts) begin
            // 考点①：随机选择突发类型（覆盖全部 8 种）
            void'(std::randomize(chosen_burst) with {
                chosen_burst inside {allowed_bursts};
            });

            // NONSEQ 首拍
            req = ahb_burst_item::type_id::create("ahb_first");
            start_item(req);
            if (!req.randomize() with {
                hburst     == chosen_burst;
                htrans     == ahb_burst_item::NONSEQ;
                hsize      == ahb_burst_item::WORD;
                busy_slots == 0;
            }) `uvm_fatal("RAND_FAIL", "AHB first beat rand failed")
            finish_item(req);
            get_response(rsp);

            beats      = req.burst_beats();
            exp_addr   = req.haddr;
            total_wait = rsp.wait_cycles;
            if (rsp.hresp) `uvm_error("AHB_BURST", "HRESP=ERROR on first beat")

            // 考点②：SEQ 拍 + 随机位置插入 BUSY
            busy_remain = req.busy_slots;
            for (int beat = 1; beat < beats; beat++) begin
                if (busy_remain > 0 && $urandom_range(0, 1)) begin
                    req = ahb_burst_item::type_id::create("ahb_busy");
                    start_item(req);
                    req.htrans = ahb_burst_item::BUSY;
                    req.hburst = chosen_burst;
```

```
                    finish_item(req);
                    get_response(rsp);
                    busy_remain--;
                end

                req = ahb_burst_item::type_id::create($sformatf("ahb_seq%0d", beat));
                start_item(req);
                if (!req.randomize() with {
                    hburst == chosen_burst;
                    htrans == ahb_burst_item::SEQ;
                    hsize  == ahb_burst_item::WORD;
                    haddr  == req.beat_addr(beat, exp_addr);
                }) `uvm_fatal("RAND_FAIL", "AHB SEQ beat rand failed")
                finish_item(req);
                get_response(rsp);

                // 考点③: HREADY 等待周期统计
                total_wait += rsp.wait_cycles;
                if (rsp.hresp) begin
                    `uvm_error("AHB_BURST",
                        $sformatf("HRESP=ERROR at beat %0d, aborting", beat))
                    break;
                end
            end

            `uvm_info("AHB_BURST",
                $sformatf("BURST type=%-6s beats=%0d total_wait=%0d",
                    chosen_burst.name(), beats, total_wait), UVM_LOW)
        end
    endtask
endclass
```

题目 24　APB4 全特性覆盖（PSTRB / PPROT / PREADY / PSLVERR 四阶段）

难度：高级
标签：APB4 / PSTRB / PPROT / PREADY / PSLVERR / write-then-read / 字节使能

【题目背景】

APB4（AMBA 4 APB Protocol Specification）在 APB3 基础上增加了两个信号： - PSTRB[3:0]：写字节使能，对应 32-bit 数据总线的 4 个字节； 读操作时 PSTRB 必须全为 0；写操作时至少一位为 1。 - PPROT[2:0]：保护属性，由三个 bit 组合：bit[0]=DATA/INSTR，bit[1]=SECURE/NONSECURE，bit[2]=NORMAL/PRIVILEGED。 PREADY：slave 拉低时插入等待周期，总线停留在 ACCESS 状态； 总等待周期影响系统性能，需统计监控。

【实现要点】

- send_apb() 辅助 task 封装单次 APB4 传输，参数化全部输入信号；
- APB 无 burst，每次传输独立两相，协议层严格串行；
- Phase B 是 PSTRB 的经典验证场景：只更新一个字节，验证其余字节不受影响。

【完整答案代码（含全部类定义）】

```
class apb4_transaction extends uvm_sequence_item;
    typedef struct packed {
        bit data_instr;   // 0=data, 1=instruction fetch
        bit non_secure;   // 0=secure, 1=non-secure
        bit privileged;   // 0=normal, 1=privileged
    } pprot_t;

    rand bit          pwrite;
    rand bit [31:0]   paddr;
    rand bit [31:0]   pwdata;
    rand bit [3:0]    pstrb;    // APB4 字节使能
    rand pprot_t      pprot;    // APB4 保护属性
         bit [31:0]   prdata;
         bit          pslverr;
         int          ready_wait;

    `uvm_object_utils_begin(apb4_transaction)
        `uvm_field_int(pwrite,     UVM_ALL_ON)
        `uvm_field_int(paddr,      UVM_ALL_ON)
        `uvm_field_int(pwdata,     UVM_ALL_ON)
        `uvm_field_int(pstrb,      UVM_ALL_ON)
        `uvm_field_int(prdata,     UVM_ALL_ON)
        `uvm_field_int(pslverr,    UVM_ALL_ON)
        `uvm_field_int(ready_wait, UVM_ALL_ON)
    `uvm_object_utils_end

    constraint addr_align_c  { paddr[1:0] == 2'b00; }
    // 考点：读操作 pstrb 必须为 0（APB4 规范）
    constraint pstrb_read_c  { !pwrite -> pstrb == 4'b0000; }
    // 考点：写操作 pstrb 至少有一个字节有效
    constraint pstrb_write_c { pwrite  -> pstrb != 4'b0000; }
    constraint addr_range_c  { paddr <= 32'hFFFF_FFFC; }
    constraint pprot_c {
        pprot.data_instr == 1'b0;
        pprot.non_secure  dist { 1'b0 := 70, 1'b1 := 30 };
        pprot.privileged  dist { 1'b0 := 60, 1'b1 := 40 };
    }

    function new(string name = "apb4_transaction");
        super.new(name);
    endfunction

    // 根据 pstrb 计算实际写入数据（屏蔽无效字节）
    function bit [31:0] masked_wdata();
        bit [31:0] m;
        foreach (pstrb[i])
            m[i*8 +: 8] = pstrb[i] ? pwdata[i*8 +: 8] : 8'h00;
        return m;
    endfunction
endclass

class apb4_full_sequence extends uvm_sequence#(apb4_transaction);
    `uvm_object_utils(apb4_full_sequence)

    int unsigned num_random   = 16;
    int unsigned num_directed = 8;

    function new(string name = "apb4_full_sequence");
        super.new(name);
    endfunction

    // 辅助 task：发送单次 APB4 传输
    // APB 协议：driver 内部实现 SETUP→ACCESS，PREADY=0 时自动延伸
    task send_apb(
        input  bit          pwrite,
        input  bit [31:0]   paddr,
        input  bit [31:0]   pwdata,
        input  bit [3:0]    pstrb,
        input  apb4_transaction::pprot_t pprot,
        output apb4_transaction rsp_out
    );
        apb4_transaction req;
```

```
        req = apb4_transaction::type_id::create("apb4_txn");
        start_item(req);
        req.pwrite = pwrite;
        req.paddr  = paddr;
        req.pwdata = pwdata;
        req.pstrb  = pstrb;
        req.pprot  = pprot;
        finish_item(req);
        get_response(rsp_out);
    endtask

    task body();
        apb4_transaction req, rsp;
        apb4_transaction::pprot_t default_pprot;
        bit [31:0] wr_addr, wr_data;
        int total_wait = 0;

        default_pprot = '{data_instr: 0, non_secure: 1, privileged: 0};

        // Phase A: 随机读写，统计 PREADY 等待
        `uvm_info("APB4_SEQ", "=== Phase A: Random R/W ===", UVM_LOW)
        repeat (num_random) begin
            req = apb4_transaction::type_id::create("apb4_rand");
            start_item(req);
            if (!req.randomize())
                `uvm_fatal("RAND_FAIL", "APB4 rand failed")
            finish_item(req);
            get_response(rsp);
            // 考点：累计 PREADY 等待周期
            total_wait += rsp.ready_wait;
            if (rsp.pslverr)
                `uvm_error("APB4_SEQ",
                    $sformatf("Unexpected PSLVERR addr=0x%08h write=%0b",
                        rsp.paddr, rsp.pwrite))
        end
        `uvm_info("APB4_SEQ",
            $sformatf("Phase A done. Total PREADY wait=%0d", total_wait), UVM_LOW)

        // Phase B: PSTRB 字节使能——只更新 byte1，其余字节不变
        `uvm_info("APB4_SEQ", "=== Phase B: PSTRB Byte-Enable ===", UVM_LOW)
        begin
            bit [31:0] orig_val, read_back;
            bit [7:0]  new_byte1;
            wr_addr = 32'h0000_0100;

            // 读出原始值
            send_apb(0, wr_addr, 0, 4'b0000, default_pprot, rsp);
            orig_val = rsp.prdata;

            // 考点：只写 byte1 (pstrb=4'b0010) ，其余字节保持
            new_byte1 = $urandom();
            send_apb(1, wr_addr,
                {orig_val[31:16], new_byte1, orig_val[7:0]},
                4'b0010, default_pprot, rsp);
            if (rsp.pslverr)
                `uvm_error("APB4_SEQ", "PSLVERR on PSTRB write")

            // 回读验证
            send_apb(0, wr_addr, 0, 4'b0000, default_pprot, rsp);
            read_back = rsp.prdata;
            if (read_back[15:8] !== new_byte1)
                `uvm_error("APB4_SEQ",
                    $sformatf("PSTRB fail: exp byte1=0x%02h got=0x%02h",
                        new_byte1, read_back[15:8]))
            else
                `uvm_info("APB4_SEQ", "PSTRB OK", UVM_MEDIUM)
        end

        // Phase C: PPROT 四种安全属性组合覆盖
        `uvm_info("APB4_SEQ", "=== Phase C: PPROT Coverage ===", UVM_LOW)
        begin
            apb4_transaction::pprot_t pp_cases[4];
            pp_cases[0] = '{data_instr:0, non_secure:0, privileged:0}; // secure, normal
```

```
            pp_cases[1] = '{data_instr:0, non_secure:0, privileged:1}; // secure, priv
            pp_cases[2] = '{data_instr:0, non_secure:1, privileged:0}; // nonsec, normal
            pp_cases[3] = '{data_instr:0, non_secure:1, privileged:1}; // nonsec, priv
            foreach (pp_cases[i]) begin
                wr_addr = 32'h0000_0200 + i * 4;
                send_apb(1, wr_addr, $urandom(), 4'b1111, pp_cases[i], rsp);
                `uvm_info("APB4_SEQ",
                    $sformatf("PPROT[%0d] nonsec=%0b priv=%0b pslverr=%0b",
                        i, pp_cases[i].non_secure,
                        pp_cases[i].privileged, rsp.pslverr),
                    UVM_MEDIUM)
            end
        end

        // Phase D: 写后读回校验
        `uvm_info("APB4_SEQ", "=== Phase D: Write-then-Read-Back ===", UVM_LOW)
        repeat (num_directed) begin
            void'(std::randomize(wr_addr, wr_data) with {
                wr_addr[1:0] == 2'b00;
                wr_addr inside {[32'h0000_0400 : 32'h0000_07FC]};
            });

            // WRITE
            send_apb(1, wr_addr, wr_data, 4'b1111, default_pprot, rsp);
            if (rsp.pslverr) begin
                `uvm_error("APB4_SEQ",
                    $sformatf("PSLVERR on WRITE addr=0x%08h", wr_addr))
                continue;
            end

            // READ BACK (APB 无 burst，上一传输完成后才能发起下一传输)
            send_apb(0, wr_addr, 0, 4'b0000, default_pprot, rsp);
            if (rsp.pslverr) begin
                `uvm_error("APB4_SEQ",
                    $sformatf("PSLVERR on READ addr=0x%08h", wr_addr))
                continue;
            end

            // 考点: PREADY 等待不影响数据正确性
            if (rsp.prdata !== wr_data)
                `uvm_error("APB4_SEQ",
                    $sformatf("Mismatch addr=0x%08h: wrote=0x%08h read=0x%08h wait=%0d",
                        wr_addr, wr_data, rsp.prdata, rsp.ready_wait))
            else
                `uvm_info("APB4_SEQ",
                    $sformatf("W->R OK addr=0x%08h data=0x%08h wait=%0d",
                        wr_addr, wr_data, rsp.ready_wait), UVM_MEDIUM)
        end

        `uvm_info("APB4_SEQ", "APB4 full sequence complete.", UVM_LOW)
    endtask
endclass


/
```